

Perception-LIDAR Based Detection

} task-1

Ground Removal

RANSAC (Random Sample Consensus)

if (point.z < threshold)
delete point

assume the sensor to
be at a fixed height
from ground
Δ flat plane

logic: choose 3 random points to form
a temporary plane

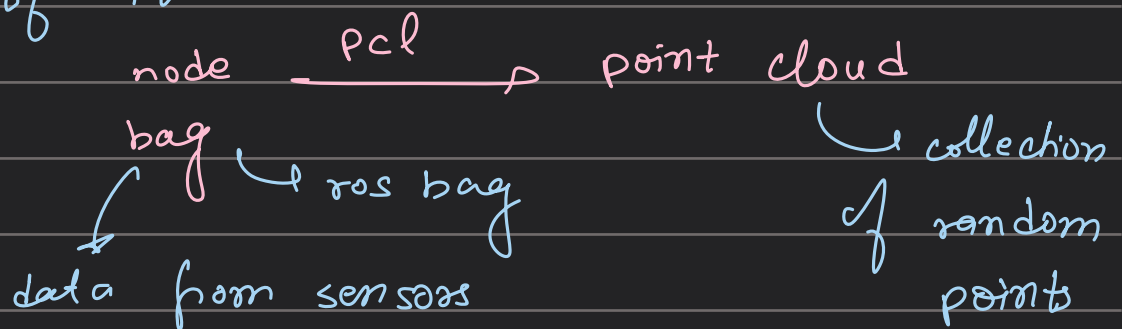
$$Ax + By + Cz = D$$

find the no. of point which lie on or in ϵ of
the plane (in Liners)

do K times the plane with most no. of
inliners is our ground
plane remove it

$$K = \frac{\log(1-p)}{\log(1-(1-e)^s)}$$

pcl: RANSAC segmentation algo takes a point cloud in
as input and finds planes such that all their points
are in ϵ of it.



PCL: Point Cloud Library

c++ library used for
point cloud processing

provides ready to use

implementations for RANSAC algo

Working Logic: consider any random 3 points to form a temporary plane: $Ax + By + Cz + D = 0$

(x_1, y_1, z_1)

for any point into consideration we calculate the $\perp^r$ distance of the point from the temp. plane by
$$d = \frac{|Ax_1 + By_1 + Cz_1 + D|}{(A^2 + B^2 + C^2)^{1/2}}$$
 & define $\epsilon \Rightarrow dToll$ (tolerance)

if $(d \leq dToll)$ $\hookrightarrow$ inliers else outliers delete them
 $\hookrightarrow$ remember the no. of inliers

Repeat this process K times until you find the plane having max. no. of inliers
 $\hookrightarrow$ ground plane which is to be removed.

$\hookrightarrow$ Task - 2: identification of what the object is

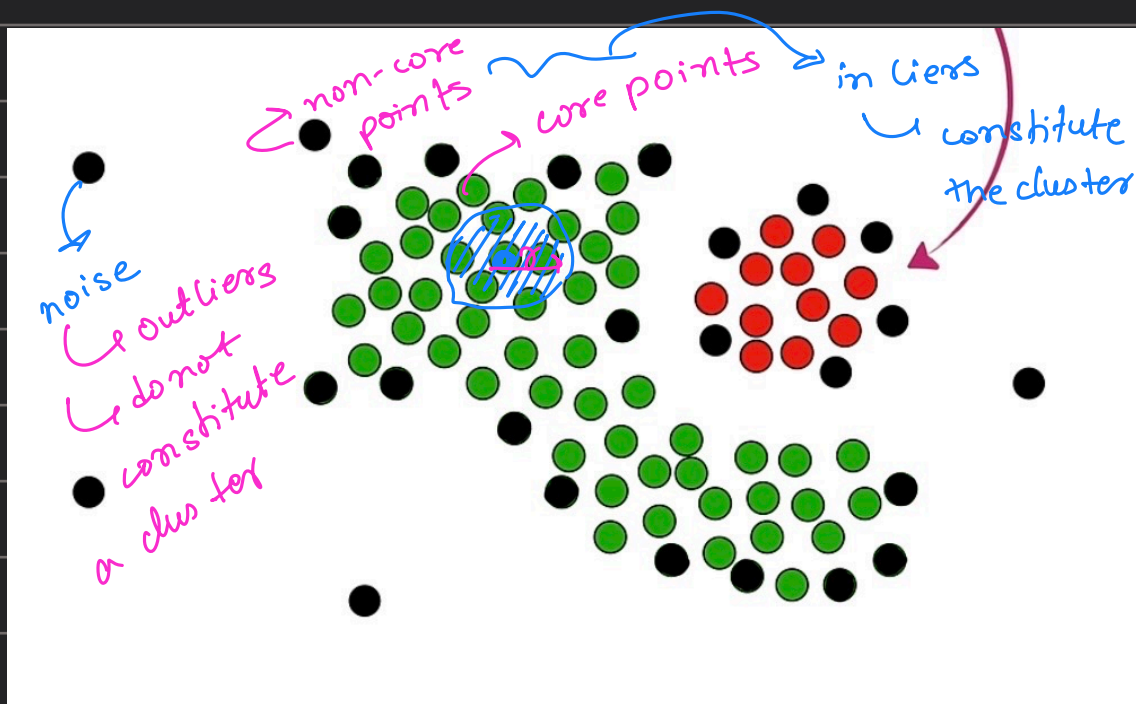
clustering

DBSCAN (Density based Spatial clustering)

Core points: Points with many neighbours

Border points: Points near the cluster but with few neighbours

Noise: Points with no neighbours $\hookrightarrow$ delete them.



Working logic: we choose a point & count the no. of points it touches for a particular radius $\hookrightarrow$ 8 points (1)

$\hookrightarrow$ do this for all the points

Now we define the min no. of points to be in contact with in order to be called a core point

$\hookrightarrow$ we found out our core points

& the remaining points which had contact with 1 or less than the min no. of points are considered as non-core points

Core points can be used for extension of the cluster but the points which are in contact with the non-core points will not be part of that cluster thus the non-core points cannot cause extension in the size of the cluster

→ adjectives of an object → colour
classification of cones (blue-white-blue
vs yellow-black-yellow)

we have two different types of cones

blue-white-blue

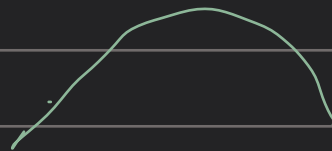
on the basis of intensity

→ high-low-high

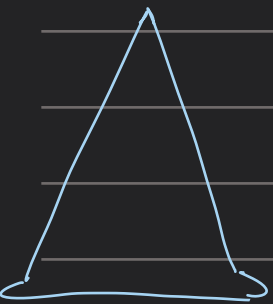
yellow-black-yellow

: on the basis of intensity

→ low-high-low



we can imagine the curve like a parabola



$$I(z) = az^2 + bz + c$$

$a > 0$

blue cone

$a < 0$

yellow cone

Task-4

you have been given a ros bag (node bag) in Point-Cloud which consists of LIDAR data for a set of cones

- (1) remove the ground using ransac

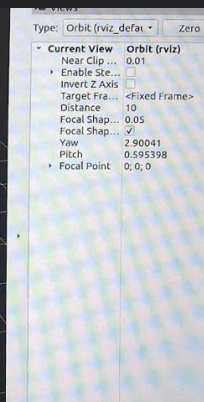
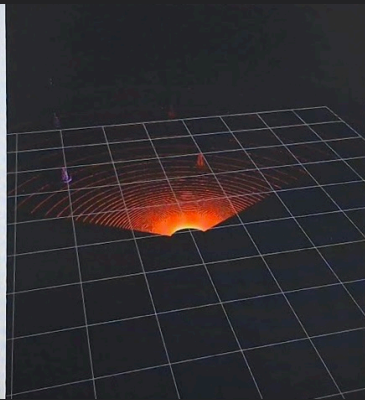
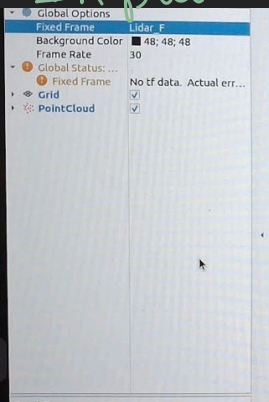
- (2) Identify the cluster & find there centroid

using DBSCAN

- (3) Using the Quad relation b/w z & I & classify the blue & yellow markers

- (4) Publish them in rviz2

Input

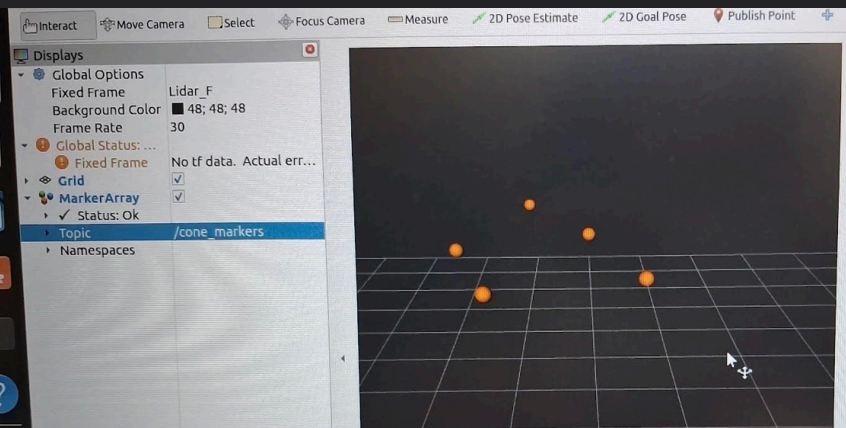


fixed frame:

Lidar_F

Add by topic

↳ /point cloud

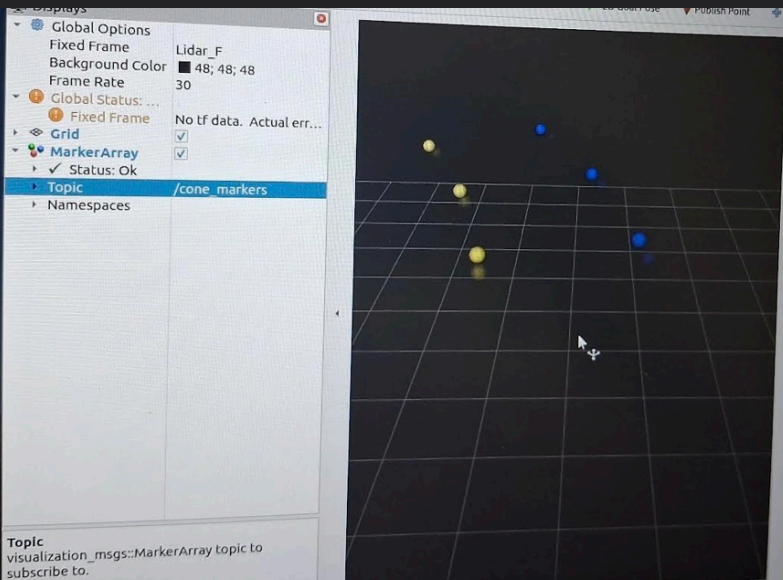


fixed frame:

Lidar_F

Add: markerArray

↳ /cone-markers



fixed frame

Lidar_F

Add: markerArray

Output

↳ /cone-marker

c> perception_package > src > ground_points_fm_and_clustering.cpp > ConeDetector

```
1 #include <rclcpp/rclcpp.hpp>
2 #include <sensor_msgs/msg/point_cloud.hpp>
3 #include <visualization_msgs/msg/marker_array.hpp>
4 #include <geometry_msgs/msg/point32.hpp>
5 #include <sensor_msgs/msg/channel_float32.hpp>
6
7 #include <vector>
8 #include <cmath>
9 #include <cstdlib>
10 #include <ctime>
```

used to publish markers in RViz
defines a 32 bit floating point coord system
used for additional channels like intensity (x,y,z)

```
12 struct Point
13 {
14     float x, y, z, intensity;
15 };
16
```

```
17 struct Plane
18 {
19     float a, b, c, d;
20 };
21
```

class ConeDetector : public rclcpp::Node ~ constructor of the class Node

```
22
23
24 public:
25     ConeDetector() : Node("cone_detector") Name of the class node is "cone_detector"
```

```
26 {
27     sub_ = this->create_subscription<sensor_msgs::msg::PointCloud>(
28         "/carmaker/pointcloud",
29         rclcpp::SensorDataQoS(),
30         std::bind(&ConeDetector::callback, this, std::placeholders::_1));
```

```
31
32
33     marker_pub_ = this->create_publisher<visualization_msgs::msg::MarkerArray>(
34         "/cone_markers", 10);
35     std::srand(std::time(nullptr)); // initialises random generator
```

```
36     std::srand(std::time(nullptr)); // needed for ransac
37
38     RCLCPP_INFO(this->get_logger(), "Cone Detector Node Started");
39 }
```

RCL-CPP INFO
RCL-CPP WARN
RCL-CPP ERROR
logging

#include <rclcpp/rclcpp.hpp> ~ import ROS2 c++ client libraries
& provides classes like : Node,

#include <sensor_msgs/msg/point_cloud.hpp> Pub, Sub, logg,

~ import PointCloud message types which consists of points
& channels

we have defined 2 structures ~ Point: for coordinates
plane: for coefficients
 $ax + by + cz + d = 0$

Subscription creation

~ create a subscriber which listens to the LIDAR messages
& the name of the topic for this subscriber is "/carmaker/point Cloud"

Marker publisher:

~ creates a publisher that sends cone markers
the topic visible in RViz is "/cone_markers"

```

private:
    rclcpp::Subscription<sensor_msgs::msg::PointCloud>::SharedPtr sub_; → stores subscriber pointer
    rclcpp::Publisher<visualization_msgs::msg::MarkerArray>::SharedPtr marker_pub_; → stores publisher pointer
    std::vector<int> best_inliers; → a vector is create to store the good points which
    are maximum in no.

std::vector<Point> convertCloud(const sensor_msgs::msg::PointCloud::SharedPtr msg)
{
    std::vector<Point> cloud;

    int intensity_channel = -1;

    for (size_t c = 0; c < msg->channels.size(); c++)
    {
        if (msg->channels[c].name == "intensity")
        {
            intensity_channel = c;
            break;
        }
    }

    for (size_t i = 0; i < msg->points.size(); i++)
    {
        Point p;

        p.x = msg->points[i].x;
        p.y = msg->points[i].y;
        p.z = msg->points[i].z;

        if (intensity_channel != -1 &&
            msg->channels[intensity_channel].values.size() > i)
        {
            p.intensity = msg->channels[intensity_channel].values[i];
        }
        else
        {
            p.intensity = 0.0;
        }

        cloud.push_back(p);
    }

    return cloud;
}

```

for finding the channel containing LiDAR intensity

for finding all the points & storing them in vector cloud

extract intensity

std::vector<Point> convertCloud(const sensor_msgs::msg::PointCloud::SharedPtr msg)

↳ converts ROS PointCloud to a vector named cloud

```
Plane computePlane(Point p1, Point p2, Point p3)
```

```
{
    float v1x = p2.x - p1.x;
    float v1y = p2.y - p1.y;
    float v1z = p2.z - p1.z;
```

a member fxn which returns the variables of the defined struct.

```
float v2x = p3.x - p1.x;
float v2y = p3.y - p1.y;
float v2z = p3.z - p1.z;
```

compute vectors

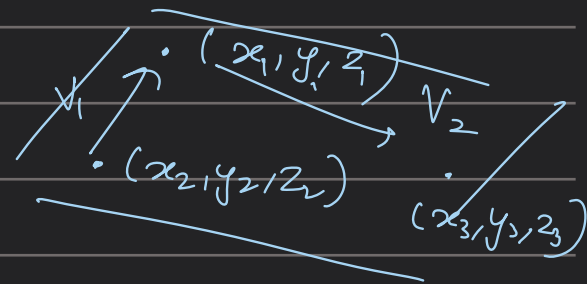
```
float a = v1y * v2z - v1z * v2y;
float b = v1z * v2x - v1x * v2z;
float c = v1x * v2y - v1y * v2x;
float d = -(a * p1.x + b * p1.y + c * p1.z);
```

finding the coefficients

```
return {a, b, c, d};
}
```

1) Create two vectors v_1 & v_2

2) Using $v_1 \times v_2$ find the normal vector which are coeff a, b, c



3) find d by substituting any point in the eqⁿ of the plane

```
float distanceToPlane(Point p, Plane plane)
```

```
{
    return std::fabs(
        plane.a * p.x + plane.b * p.y + plane.c * p.z + plane.d) /
        std::sqrt(
            plane.a * plane.a + plane.b * plane.b + plane.c * plane.c);
}
```

input argument are two structures

$$\text{dist} = \frac{|ax_1 + by_1 + cz_1|}{(a^2 + b^2 + c^2)^{1/2}}$$


```
Plane ransac(std::vector<Point> &cloud)
```

```
{
```

```
RCLCPP_INFO(this->get_logger(), "Running RANSAC on %ld points", cloud.size());
```

```
best_inliers.clear();
```

```
int max_iterations = 200;
```

```
float threshold = 0.02;  $\epsilon$ 
```

```
Plane best_plane;
```

```
for (int i = 0; i < max_iterations; i++)
```

```
{
```

```
int i1 = rand() % cloud.size();
```

```
int i2 = rand() % cloud.size();
```

```
int i3 = rand() % cloud.size();
```

```
if (i1 == i2 || i2 == i3 || i1 == i3)
```

```
continue;
```

```
Plane plane = computePlane(cloud[i1], cloud[i2], cloud[i3]);
```

```
std::vector<int> current_inliers;
```

```
for (int i = 0; i < cloud.size(); i++)
```

```
{
```

```
float dist = distanceToPlane(cloud[i], plane);
```

```
if (dist < threshold)
```

```
current_inliers.push_back(i);
```

```
}
```

```
if (current_inliers.size() > best_inliers.size())
```

```
{
```

```
best_inliers = current_inliers;
```

```
best_plane = plane;
```

```
}
```

```
RCLCPP_INFO(this->get_logger(), "RANSAC finished. Inliers: %ld", best_inliers.size());
```

```
return best_plane;
```

```
}
```

printing the points on which ransac is working

we have to compute the plane which best resembles the ground by finding the ground which has max. no. of inliers.

picking of 3 random points

run the loop in the entire cloud vector

include all these points

re occurrence

print the inliers

of the best plane

we randomly choose 3 unique points $cloud[i_1]$, $cloud[i_2]$, $cloud[i_3]$ and compute a plane from these points

the calculate the dist of $cloud[i]$ from the temporary plane i times

repeat this i times to find the plane having max no. of inliers

```
std::vector<std::vector<int>> dbscan(std::vector<Point> &cloud)
```

```
{
    float eps = 0.2; // neighbour hood radius
    int min_pts = 8; // min no. of points required to form a cluster

    std::vector<bool> visited(cloud.size(), false);
    std::vector<std::vector<int>> clusters; // process the point

    for (size_t i = 0; i < cloud.size(); i++) // store clusters
    {
        if (visited[i]) // already part of some other cluster
            continue;
        visited[i] = true; // marking the point as visited

        std::vector<int> neighbors = regionQuery(cloud, i, eps); // finds all the points within eps

        if (neighbors.size() < min_pts)
            continue;

        std::vector<int> cluster;
        cluster.push_back(i);

        for (int j = 0; j < neighbors.size(); j++)
        {
            int n = neighbors[j]; // n represent that point in the vector
            // neighbours which is a neighbour of
            // the chosen point at the index j

            if (!visited[n])
            {
                visited[n] = true;

                auto new_neighbors = regionQuery(cloud, n, eps); // check for neighbours of
                // the neighbour
                if (new_neighbors.size() >= min_pts)
                    neighbors.insert(neighbors.end(), new_neighbors.begin(), new_neighbors.end());
            }

            cluster.push_back(n); // add the point to the cluster
        }

        clusters.push_back(cluster); // add the cluster to clusters
    }

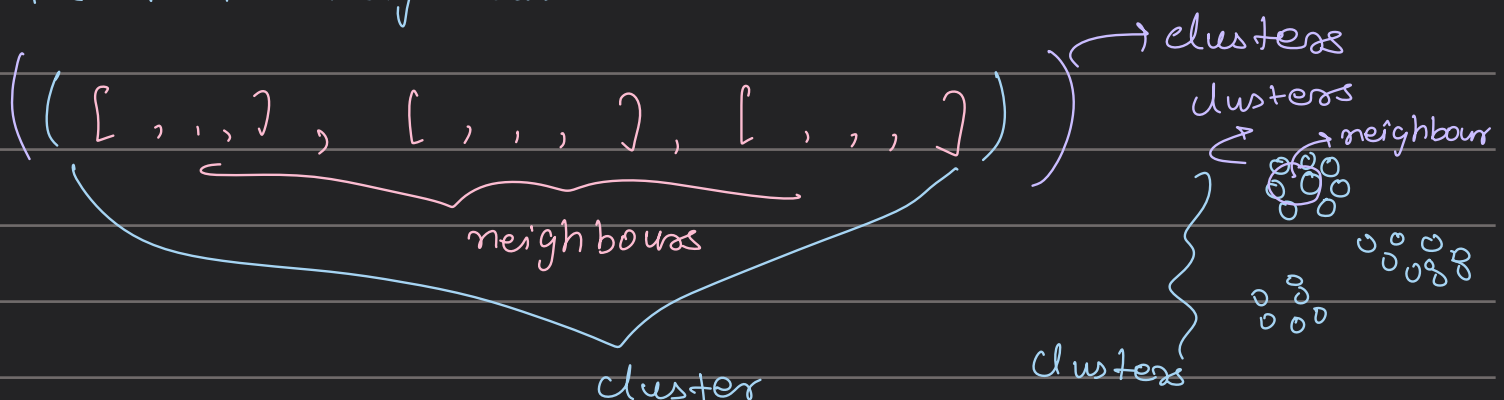
    RCLCPP_INFO(this->get_logger(), "DBSCAN found %ld clusters", clusters.size());

    return clusters;
}
```

Algorithm: pick point $\rightarrow$ find neighbours $\rightarrow$ expand cluster
 $\downarrow$
 repeat

we first track the core points

we choose any non-visited point from the vector cloud & use fn `regionQuery()` to find all the points which can form cluster with it & stores it in the vector neighbours



```
std::vector<int> regionQuery(std::vector<Point> &cloud, int index, float eps)
```

```
{
    std::vector<int> neighbors;

    for (size_t i = 0; i < cloud.size(); i++)
    {
        float dx = cloud[i].x - cloud[index].x;
        float dy = cloud[i].y - cloud[index].y;
        float dz = cloud[i].z - cloud[index].z;

        float dist = std::sqrt(dx * dx + dy * dy + dz * dz);

        if (dist < eps)
            neighbors.push_back(i);
    }

    return neighbors;
}
```

calculating the distance of
all the points to see
whether they lie within or
outside the defined radius

$$d = \sqrt{(dx)^2 + (dy)^2 + (dz)^2}$$

```
float computeQuadraticA(const std::vector<Point> &cluster_points)
```

```
{
    float sum_z = 0;
    float sum_z2 = 0;
    float sum_z3 = 0;
    float sum_z4 = 0;

    float sum_i = 0;
    float sum_zi = 0;
    float sum_z2i = 0;

    int n = cluster_points.size();

    for(const auto &p : cluster_points)
    {
        float z = p.z;
        float I = p.intensity;

        sum_z += z;
        sum_z2 += z*z;
        sum_z3 += z*z*z;
        sum_z4 += z*z*z*z;

        sum_i += I;
        sum_zi += z*I;
        sum_z2i += z*z*I;
    }

    float A[3][3] = {
        {sum_z4, sum_z3, sum_z2},
        {sum_z3, sum_z2, sum_z},
        {sum_z2, sum_z, (float)n}
    };

    float B[3] = {sum_z2i, sum_zi, sum_i};

    float detA =
        A[0][0]*(A[1][1]*A[2][2] - A[1][2]*A[2][1]) -
        A[0][1]*(A[1][0]*A[2][2] - A[1][2]*A[2][0]) +
        A[0][2]*(A[1][0]*A[2][1] - A[1][1]*A[2][0]);

    if(std::fabs(detA) < 1e-6)
        return 0;

    float detA1 =
        B[0]*(A[1][1]*A[2][2] - A[1][2]*A[2][1]) -
        A[0][1]*(B[1]*A[2][2] - A[1][2]*B[2]) +
        A[0][2]*(B[1]*A[2][1] - A[1][1]*B[2]);

    float a = detA1 / detA;

    return a;
}
```

finding the quad. poly ax^2+bx+c
for intensity relations

$$I(z) = az^2 + bz + c$$

Build normal matrix
 $Ax=B$

and find the coeff. using Cramer's rule

construct a regression matrix

construct a 2nd matrix

calculating $|A|$

calculating $|A_1|$

$$a = \frac{|A_1|}{|A|}$$

Using Linear regression :-

find $\sum z$, $\sum z^2$, $\sum z^3$, $\sum z^4$, $\sum \bar{z}$, $\sum z\bar{z}$, $\sum z^2\bar{z}$ & make a 3×3 regression matrix

$$A = \begin{bmatrix} \sum z^4 & \sum z^3 & \sum z^2 \\ \sum z^3 & \sum z^2 & \sum z \\ \sum z^2 & \sum z & n \end{bmatrix}$$

n : no. of points in a cluster

Solving the eqⁿ

$Ax = B$ using Cramer's rule

$$B = \begin{bmatrix} \sum z^2\bar{z} \\ \sum z\bar{z} \\ \sum \bar{z} \end{bmatrix}$$

$$a = \frac{|A|}{|A|}$$

```
void callback(const sensor_msgs::msg::PointCloud::SharedPtr msg)
{
```

```
    RCLCPP_INFO(this->get_logger(), "PointCloud message received");
    auto cloud = convertCloud(msg);
```

```
    RCLCPP_INFO(this->get_logger(), "Converted cloud size: %ld", cloud.size());
```

```
    if (cloud.size() < 50)
```

```
    {
        RCLCPP_WARN(this->get_logger(), "Point cloud too small");
        return;
    }
```

↳ triggered when Point Cloud message arrives

```
    RCLCPP_INFO(this->get_logger(), "Starting RANSAC ground removal");
```

```
    Plane ground_plane = ransac(cloud);
```

```
    std::vector<bool> is_ground(cloud.size(), false);
```

```
    for (auto idx : best_inliers)
        is_ground[idx] = true;
```

```
    std::vector<Point> cloud_no_ground;
```

↳ checking min no. of points in the vector point cloud

```
    for (int i = 0; i < cloud.size(); i++)
```

```
    {
        if (!is_ground[i])
            cloud_no_ground.push_back(cloud[i]);
    }
```

```
    RCLCPP_INFO(this->get_logger(), "Non-ground points: %ld", cloud_no_ground.size());
```

```
    RCLCPP_INFO(this->get_logger(), "Running DBSCAN clustering");
```

```
    auto clusters = dbscan(cloud_no_ground);
```

③

①: call convertCloud fxn

②: call ransac algo

③: call dbscan algo

```
visualization_msgs::msg::MarkerArray marker_array;
```

→ create marker_array

```
visualization_msgs::msg::Marker delete_marker;
```

→ to delete old markers

```
delete_marker.action = visualization_msgs::msg::Marker::DELETEALL;
```

```
marker_array.markers.push_back(delete_marker);
```

```
int id = 0;
```

```
for (auto &cluster : clusters)
```

```
{
```

```
RCLCPP_INFO(this->get_logger(), "Processing cluster with %ld points", cluster.size());
```

```
float cx = 0, cy = 0, cz = 0;
```

```
for (int idx : cluster)
```

```
{
```

```
cx += cloud_no_ground[idx].x;
```

```
cy += cloud_no_ground[idx].y;
```

```
cz += cloud_no_ground[idx].z;
```

```
}
```

```
cx /= cluster.size();
```

```
cy /= cluster.size();
```

```
cz /= cluster.size();
```

} compute centroid (cx, cy, cz) of each of the clusters

```
std::vector<Point> cluster_points;
```

```
for (int idx : cluster){
```

```
cluster_points.push_back(cloud_no_ground[idx]);
```

```
}
```

```
float a = computeQuadraticA(cluster_points);
```

```
RCLCPP_INFO(this->get_logger(), "Polynomial coefficient a = %f", a);
```

```
visualization_msgs::msg::Marker marker;
```

```
marker.header.frame_id = "Lidar_F";
```

```
marker.header.stamp = this->get_clock()->now();
```

```
marker.ns = "cones";
```

```
marker.id = id++;
```

```
marker.type = visualization_msgs::msg::Marker::SPHERE;
```

```
marker.action = visualization_msgs::msg::Marker::ADD;
```

→ sphere shaped markers

$$cx = \frac{\sum x_i}{N}, \quad cy = \frac{\sum y_i}{N}, \quad cz = \frac{\sum z_i}{N}$$

→ cluster.size()

sum of
x-coordinates of
all the points of a cluster

```

        marker.pose.position.x = cx;
        marker.pose.position.y = cy;
        marker.pose.position.z = cz;

        marker.scale.x = 0.3;
        marker.scale.y = 0.3;
        marker.scale.z = 0.3;

        float threshold = 1e-5;
        if(a > threshold){
            marker.color.r = 0.0;
            marker.color.g = 0.0;
            marker.color.b = 1.0;
        }
        else if(a<-threshold){

            marker.color.r = 1.0;
            marker.color.g = 1.0;
            marker.color.b = 0.0;
        }

        marker.color.a = 1.0;

        marker.lifetime = rclcpp::Duration::from_seconds(0);

        marker_array.markers.push_back(marker);
    }

    RCLCPP_INFO(this->get_logger(), "Publishing %ld cone markers", marker_array.markers.size());

    marker_pub->publish(marker_array);
    RCLCPP_INFO(this->get_logger(), "Markers published successfully");
}

};

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    RCLCPP_INFO(rclcpp::get_logger("cone_detector"), "Starting node...");
    rclcpp::spin(std::make_shared<ConeDetector>());
    rclcpp::shutdown();
    return 0;
}

```

